

- Lattice ECP5 (Project Trellis ecosystem; more capable, still open)

And we'll use the most common open flow:

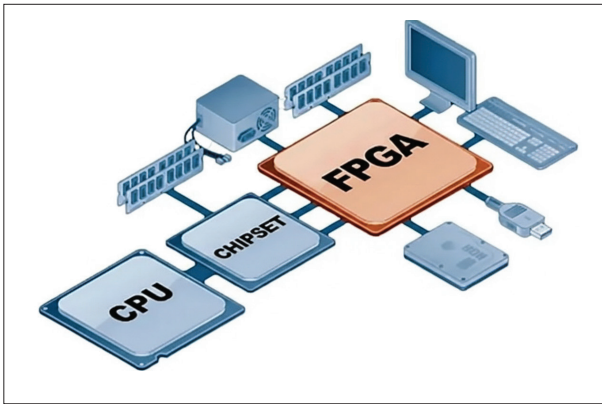
- Yosys for Verilog RTL synthesis
- nextpnr for FPGA place and route
- IceStorm / Trellis tooling to emit a real device bitstream

FPGA in one page: what is it?

An FPGA (Field-Programmable Gate Array) is a chip you can reconfigure after manufacturing. Instead of fixed circuits like an MCU or ASIC, an FPGA contains programmable logic blocks and routing that you “wire up” by loading a configuration bitstream.

Why FPGAs are perfect for learners:

- You design with HDL (like Verilog)
- You get real timing, pin mapping, and hardware behavior
- You can iterate quickly without fab costs



A clean way to remember the FPGA value proposition: it's a hardware sandbox – fast iteration, hardware realism, low financial risk. (Architecture primers explain the same big blocks: logic blocks, interconnect, I/O, memory/DSP)

Inside an FPGA

Every FPGA family has its own details, but the “city plan” is consistent:

Logic blocks (CLBs / logic cells)

These contain LUTs (Look-Up Tables) + flip-flops.

A LUT is basically a tiny truth-table engine: it can implement arbitrary boolean logic up to a certain input size. Different families use different LUT sizes (iCE40 commonly uses smaller LUT structures; many modern FPGAs use 6-input LUTs). Don't obsess over LUT size – obsess over the concept: LUTs are the configurable “logic fabric.”

Programmable interconnect

This is the routing “road network” between logic blocks. Place & route tools decide which roads to use to meet timing.

I/O blocks

The interface between FPGA logic and real pins. Voltage standards, drive strength, pull-ups – this is where “it compiles” becomes “it works.”

BRAM (Block RAM)

Dedicated memory blocks on-chip (far more efficient than building memory out of LUTs). Used for buffers, FIFOs, frame-buffers, lookup tables, tiny CPU memories.

DSP blocks

Dedicated multiply-accumulate hardware (think: signal processing, filters, ML-ish math). ECP5, for instance, has dedicated sysDSP blocks, documented in vendor guides.

Remember: LUTs do logic, flip-flops hold state, interconnect moves signals, I/O touches reality, BRAM stores, DSPs multiply fast.

FPGA vs ASIC vs microcontroller

A microcontroller is great when:

- you need software flexibility
- you're controlling peripherals and doing moderate computation
- determinism matters but “hardware parallelism” isn't critical

An FPGA is great when:

- you need true parallelism (many operations at once)
- you need precise, deterministic timing at hardware level
- you want to prototype custom hardware blocks (UARTs, video pipelines, custom accelerators)
- you're learning chip design fundamentals without ASIC costs

An ASIC is great when:

- you want the lowest power / highest performance / lowest cost at huge volume
- you can afford verification effort and manufacturing complexity

For this Fast Track: FPGA is the sweet spot. You'll learn the real rules (timing, constraints, state machines) while still having fun.

Your first board: what to look for (and what to ignore)

For the purpose of this chapter, your board checklist is simple:

Must-haves

- At least one user LED (preferably more)
- At least one user button
- A reliable programming path (USB or JTAG adapter support)
- A clear pinout / constraints file reference (PCF for iCE40, LPF for ECP5)

Nice-to-haves

- Onboard oscillator (clock)
- UART bridge to USB (easy serial debug later)